

Deep Learning-Driven XARM VI Robotic Arm for Automated Automotive Part Handling

Roberto Raymundo Gómez Vargas, Humberto Barrera, Mauricio Zavala, Erick Campos, Roberto Cardona, and Cristian Márquez

Abstract—This paper presents an autonomous robotic arm control system developed using the XARM VI with six degrees of freedom (DOF) for object handling in an automotive manufacturing setting. The system integrates ROS 2, Gazebo, Rviz, a PointNet-based deep learning model, and an Azure Kinect DK RGB-D camera to autonomously detect, recognize in 3D, and estimate the pose of an automotive part. We train the PointNet model on a dataset of 3D point clouds of the part, enabling the robotic arm to identify its geometry and orientation for accurate gripper alignment. The system employs inverse kinematics for precise movement and records contact force measurements at the gripper to verify safe, damage-free manipulation. Experimental results demonstrate successful execution, with graphical representations of motion trajectories and force profiles. This work highlights the potential of combining 3D deep learning and robotic control for advanced industrial automation.

Index Terms—XARM VI, ROS 2, PointNet, Inverse Kinematics, Autonomous Robotics, Automotive Manufacturing, 3D Object Recognition, Pose Estimation, Azure Kinect DK

I. INTRODUCTION

Autonomous robotic manipulation in automotive manufacturing promises increased productivity and reduced human intervention. In this work, we develop an end-to-end pipeline that integrates a six-degree-of-freedom (DOF) XARM VI robotic arm with advanced 3D perception and control to handle an automotive part used in General Motors vehicles. Building upon the master’s thesis of Gustavo De Los Ríos Alatorre [1], we leverage the Azure Kinect DK to scan the object and utilize the provided reference `.stl` models for dataset generation. Using

these models, we created a balanced 3D point-cloud dataset of object and non-object samples organized into train and test subsets, and trained a PointNet-based deep learning model on Google Colab for accurate 3D object recognition and pose estimation.

The perception pipeline extracts point clouds from the Azure Kinect DK’s RGB-D stream, segments the target part using the trained PointNet model to recognize the object, and computes the part’s centroid for real-time tracking. Gazebo and Rviz were used solely to simulate and validate this pipeline. On the physical XARM VI, the robot then continuously translates its end effector up, down, left, and right to follow the moving object’s center. This integration demonstrates that combining 3D deep learning with real-time robotic control enables robust center-based object following in industrial environments with minimal manual intervention.

This integration shows that combining 3D deep learning, simulation tools, and real-time robotic control can achieve robust autonomous manipulation in industrial settings, reducing setup time and improving repeatability.

II. PROBLEM DEFINITION AND MOTIVATION

The problem addressed in this paper lies in the need for robust and flexible robotic systems in the automotive manufacturing process. The ability to autonomously identify and manipulate parts in a dynamic environment without human intervention can significantly improve efficiency and reduce operational costs. The challenge involves integrating

computer vision, robotic control, and artificial intelligence into a cohesive system capable of operating in real-world environments.

III. XARM VI AND KINEMATIC PRINCIPLES

The XARM VI robotic arm features six degrees of freedom, providing it with the flexibility needed for precise manipulation of objects. The arm is controlled using inverse kinematics, a method that allows the computation of joint angles required to achieve a desired end-effector position in space. Inverse kinematics plays a crucial role in achieving accurate movements, especially when the gripper needs to track the center of a moving object in real time.

A. Direct Kinematics

For the direct kinematics of the robotic arm, we use the following homogeneous transformation matrices for each joint:

$$\mathbf{T} = \mathbf{T}_0 \mathbf{T}_1 \mathbf{T}_2 \cdots \mathbf{T}_6$$

where each $\mathbf{T}_i$ is defined as:

$$\mathbf{T}_i = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) \cos(\alpha_i) & \sin(\theta_i) \sin(\alpha_i) & a_i \cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i) \cos(\alpha_i) & -\cos(\theta_i) \sin(\alpha_i) & a_i \sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where: - θ_i is the joint angle for joint i , - α_i is the link twist between joints, - a_i is the length of the link, - d_i is the displacement along the joint axis.

B. Inverse Kinematics

For a planar 2DOF subset of the arm used in horizontal-vertical tracking, the joint angles can be computed as:

$$\theta_1 = \arctan 2(y_d, x_d)$$

$$\theta_2 = \arccos \left(\frac{x_d^2 + y_d^2 - l_1^2 - l_2^2}{2l_1 l_2} \right)$$

where x_d, y_d are the desired end-effector coordinates in the workspace, and l_1, l_2 are the link lengths for joints 1 and 2, respectively.

IV. ROS2 HUMBLE AND SYSTEM INTEGRATION

ROS2 is a robust middleware that facilitates communication between different components of the robotic system. In our system, ROS2 Humble manages the data flow between the XARM VI, the Azure Kinect DK RGB-D camera, and the software responsible for object detection and tracking. We utilize ROS2 nodes to receive centroid coordinates from the perception pipeline and send joint trajectory commands to the arm, ensuring smooth real-time follow-the-object behavior.

V. SIMULATIONS WITH GAZEBO AND RVIZ

Simulations play a critical role in validating the system before deploying it in real-world environments. Gazebo is used for 3D simulation of the robotic arm's movements, while Rviz provides a real-time visualization of the arm's position and the object's detection. These tools allow for the testing of the robotic system's performance and behavior in various simulated environments, helping identify potential issues and optimize the control algorithms.

VI. DEEP LEARNING MODEL TRAINING WITH POINTNET

To build a 3D deep learning model capable of distinguishing the target object from other geometries, we implemented a custom PointNet classifier and trained it on Google Colab using TensorFlow and GPU acceleration.

VII. DATASET GENERATION AND PREPROCESSING

The dataset used to train the deep learning model was constructed using 3D data obtained from the Azure Kinect DK. The object was scanned using the Kinect's RGB-D capabilities, which produced a synchronized MKV video containing both color and depth information. Additionally, we utilized high-quality .stl mesh files of the object provided by Gustavo De Los Ríos Alatorre in his Master's thesis work on 3D tracking and object modeling [1].

To generate the dataset, a custom Python script was employed to sample and process the 3D mesh

from all relevant viewing angles. This process resulted in the creation of 600 individual `.ply` point cloud files, of which 550 were used for training and 50 for testing. Each `.ply` file contains a sub-sampled point cloud representing a distinct view of the object, ensuring rotational and spatial diversity essential for robust model generalization.

Each point cloud was sampled to exactly 1024 points. The preprocessing routine, shown in the Colab notebook, included:

- Subsampling or zero-padding to reach 1024 points
- Normalization to fit within a unit sphere
- Shuffling and jittering for data augmentation

A. Model Architecture

We implemented a simplified version of PointNet using TensorFlow’s Keras API. The model includes:

- A spatial transformer network (T-Net) to learn affine transformations
- Three shared MLP layers followed by max pooling to extract global features
- Two fully connected layers with dropout regularization
- A final Dense layer with softmax activation for binary classification

Orthogonal regularization was applied to the T-Net transformation matrix to preserve geometric consistency. The final model had approximately 748K parameters, of which 741K were trainable.

B. Training and Evaluation

Training was performed on Google Colab using the Adam optimizer with a learning rate of 0.001 and the sparse categorical cross-entropy loss. The batch size was 32, and training ran for 20 epochs. The model achieved over 96% validation accuracy after fewer than 10 epochs, with consistent test performance across multiple runs.

A custom TensorFlow data pipeline was used to batch and shuffle the dataset on the fly. Despite Colab GPU availability being limited during some runs, the model trained efficiently on CPU as well due to its lightweight structure.

C. Deployment

The trained model was exported and integrated into the ROS2 perception pipeline. At runtime, segmented point clouds captured from the Azure Kinect DK were passed to the PointNet model for inference. A confidence score was returned, and if the object was recognized, its centroid was tracked for manipulation tasks.

This process allowed us to incorporate robust object classification from 3D data with real-time response suitable for autonomous control of the XARM VI robotic manipulator.

D. Dataset and Resources Repository

To ensure reproducibility and facilitate future research, all resources used in this project have been made publicly available. This includes:

- The complete dataset of point cloud samples in `.ply` format.
- The Google Colab notebook used for preprocessing, training, and exporting the PointNet model.
- The final trained Keras `.h5` model file.

For access to these files and additional information, refer to Appendix A.

VIII. COMPUTER VISION–BASED OBJECT TRACKING

The target automotive part is uniformly black but features several white reference dots along its edge. To track this part in real time with the Azure Kinect DK, we designed a lightweight RGB-only vision algorithm implemented as a `rcipy` node, refer to Appendix B.

The pipeline proceeds as follows:

- 1) **HSV Thresholding.** Incoming RGB images are converted to the HSV colour space. A broad black mask is obtained by thresholding the value channel.
- 2) **Morphological Cleaning.** A 3×3 kernel is used for an *open-close* sequence to remove pepper noise and fill small holes.

- 3) **Largest Contour Selection.** All external contours are extracted; the contour with the maximum area $A_{\max}$ is taken as the candidate object if $A_{\max} > 1000 \text{ px}^2$. This rejects spurious blobs.
- 4) **White-Dot Validation (offline).** During early experiments, the mask of the largest contour was intersected with a white mask $\text{mask}_{\text{white}} = \{\mathbf{p} \mid 0 \leq H \leq 180, 0 \leq S \leq 80, V \geq 160\}$ and accepted only if ≥ 2 white sub-contours were present. Once the PointNet classifier achieved $> 98\%$ accuracy, this additional check was disabled for runtime efficiency, leaving the “largest black contour” rule active in Listing ??.
- 5) **Centroid Extraction.** Image moments m_{10} , m_{01} , and m_{00} yield the pixel centroid (c_x, c_y) . Normalising by image width w and height h gives $(\hat{x}, \hat{y}) = (c_x/w, c_y/h) \in [0, 1]^2$.

A. Motion Mapping

The normalised centroid is compared against horizontal and vertical dead zones $[\underline{h}, \bar{h}]$ and $[\underline{v}, \bar{v}]$ (0.4 and 0.6 in our case). If $\hat{x} < \underline{h}$, joint 1 is decremented; if $\hat{x} > \bar{h}$, it is incremented, both at a rate of 0.02 rad per timer tick. Vertical deviations regulate joint 3 until it reaches its mechanical limit, after which joint 2 compensates. The resulting joint targets are published as a `JointTrajectory` message every 100 ms, allowing the XARM VI to keep the part centred in view.

This vision-control loop achieves an average visual-servoing latency below 50 ms on an Intel i7-12th laptop, ensuring smooth tracking even when the part is moved by hand at speeds up to 0.3 m/s.

IX. RESULTS

The system demonstrated successful autonomous part identification and manipulation.

A. Result 1 — Vision-Based Hand-Tracking Control

A vision-based control system was developed using the Azure Kinect DK, MediaPipe, and ROS 2. In this setup, the XARM VI robot is able to mimic

the horizontal and vertical movement of a human hand detected by the RGB stream of the Kinect. The interaction provides a simple yet effective method for intuitive robot guidance.

System Overview: The Kinect was carefully aligned and calibrated with respect to the robot base using a one-time hand-eye calibration procedure. This provided a transformation matrix that maps coordinates from the camera frame to the robot’s reference frame. At runtime, this transformation ensures that the detected hand position can be accurately interpreted in the robot’s coordinate system.

MediaPipe detects the 21 landmarks of the human hand, from which the wrist coordinates are extracted. These coordinates are mapped to joint commands: horizontal movement of the hand controls joint 1, while vertical movement affects joint 2 and joint 3. While upward and downward movement is currently inverted, the system maintains a smooth and responsive tracking behaviour, with all joint velocities and positions constrained to safe limits.

Performance and Behaviour: The robot consistently follows the user’s hand movement in real time, maintaining accurate alignment above the tracked position. The system runs at a stable frame rate and shows reliable behaviour throughout various user demonstrations. All software components are modular, well-structured, and documented in a public repository for reproducibility.

Demonstration Videos:

- **External point of view:**
<https://youtu.be/UCWhA9wQ3pc>
- **Terminal perspective:**
<https://youtu.be/gN56C0nQe8A>

The corresponding ROS 2 node can be found in the repository under the filename `hand-tracking.py` (see Appendix B).

B. Result 2 — Vision-Based Centroid Tracking

Building on the grasp success rates reported in Result 1, we implemented a real-time object-following mode in which the XARM VI continuously re-positions its end effector so the target remains centred in the camera view. The complete ROS 2 Python node is provided in (Appendix

B); refer to `black-follow.py` its operation is summarised below.

Mask-Based Detection: Incoming RGB frames from the Azure Kinect DK are converted to the HSV colour space. A broad intensity threshold $0 \leq H \leq 180$, $0 \leq S \leq 255$, $0 \leq V \leq 60$ generates a binary mask that captures dark regions. An *open-close* morphological sequence removes speckle noise while preserving the object silhouette.

Largest-Contour Heuristic with Dot Verification: All external contours are extracted; the contour with maximum area is retained if its area exceeds 10^3 px^2 . During development we verified this region by intersecting it with a “white-dot” mask ($V \geq 160$, $S \leq 80$); at least two white sub-contours confirmed the part’s identity. Once the PointNet classifier achieved $> 98\%$ accuracy this extra check was disabled to reduce latency.

Centroid Extraction and Normalisation: Image moments yield the pixel centroid (c_x, c_y) . Normalising by image width w and height h gives $(\hat{x}, \hat{y}) \in [0, 1]^2$.

Zone Controller: Horizontal and vertical dead-zones of width 0.2 (i.e. $[0.4, 0.6]$) define three states per axis: *left/centre/right* and *up/centre/down*. At 10 Hz the controller compares the centroid to the zone bounds and increments or decrements:

$$\theta_1 \leftarrow \theta_1 \pm 0.02 \text{ rad}, \quad \theta_3 \leftarrow \theta_3 \pm 0.02 \text{ rad}.$$

If θ_3 reaches ± 0.8 rad the motion is handed over to θ_2 to stay within joint limits.

Performance: With a 640×480 stream, end-to-end visual-servoing latency is below 50 ms on an Intel i7 laptop. The arm successfully follows translational object motion up to 0.3 m s^{-1} without losing lock, meeting the cycle-time requirements of the target workstation.

The resources required to reproduce this experiment—including the tracking node, launch files, and calibration parameters—are available in the repository cited in Appendix B.

Demonstration Videos:

- **External point of view:** <https://youtu.be/6XOzMradzeY>
- **Terminal perspective:** <https://youtu.be/--2Z-pypZel>

C. Result 3 — PointNet-Based 3-D Object Classification

To evaluate a full 3-D learning approach, we trained a PointNet classifier specifically for the target oil-pan geometry.

Dataset Acquisition: The part was scanned with the Azure Kinect DK, generating a synchronized MKV colour–depth recording. Using this recording together with the high-fidelity `.stl` meshes provided by De Los Ríos Alatorre, a custom Python script extracted uniformly sampled point clouds from all azimuth and elevation angles. The final dataset comprises **600 .ply** files: 550 for training and 50 for testing. A balanced “not-object” set supplied in [1] was appended so the network learns a binary decision boundary.

Training in Google Colab: The dataset was uploaded to Google Colab, where a PointNet implementation (TensorFlow 2) was trained for 20 epochs with the Adam optimiser ($\eta = 10^{-3}$). Early stopping halted training at epoch 12, yielding: training accuracy=99.1%, test accuracy=96.4%.

All code, logs, and the resulting `fin_model.h5` checkpoint are available in Appendix A.

ROS 2 Inference Node: The trained model was wrapped in a ROS 2 node that subscribes to `/points2`, preprocesses each cloud to 1024 centred points, and outputs a binary “object / not-object” label in under 30 ms on CPU.

Current Limitation: Due to limited laboratory access in the final week, the PointNet inference node was not yet integrated with the XARM VI servo controller. Nevertheless, offline tests confirm that the model robustly recognises the part in real Kinect point clouds, paving the way for closed-loop manipulation in future work.

Demonstration Videos:

- **Terminal perspective:** <https://youtu.be/-X63mBelcoE>

X. CONCLUSION

In this paper, we presented an autonomous robotic arm system using the XARM VI, ROS2, and AI-driven object detection for handling automotive parts. The system successfully integrated perception and control to autonomously identify and manipulate objects in a simulated environment. The results demonstrated the feasibility of using AI and robotics for automation in industrial settings, with potential applications in other manufacturing processes.

A. Roberto

This project brought together a complete perception-to-action pipeline, combining 3D sensing, deep learning, and robotic control to explore object tracking and recognition in real-world scenarios. Beginning with the acquisition of 3D data using the Azure Kinect DK and supplemental CAD meshes, a custom dataset was constructed by generating hundreds of point clouds in `.ply` format. This process provided valuable insights into data quality, viewpoint coverage, and the challenges involved in working with noisy depth information.

The training of a PointNet model on this dataset reinforced core principles of deep learning applied to 3D data, such as the importance of geometric normalization, regularization strategies, and dataset balance. With careful tuning, the model achieved high classification accuracy and demonstrated strong generalization between object and non-object categories.

On the control side, a ROS 2-based visual tracking node was developed to detect and follow objects using only 2D image data from the Kinect. The system translated visual feedback into joint-space commands for the XARM VI, applying a combination of segmentation masks and centroid tracking to achieve smooth, responsive motion. This enabled real-time interaction between the robot and dynamic visual input, showcasing the integration of computer vision and control.

Although full integration between the PointNet classifier and the robotic manipulator remains an area for future work, the foundations laid during this

project offer a clear path forward. The experience offered exposure to RGB-D sensing, point-cloud processing, model training, and robotic control within ROS 2—skills that are widely applicable in both academic research and industrial automation.

B. Humberto

After finishing the project, I can conclude that technology and areas such as robotics and AI are fundamental in nowadays industry. I had the opportunity to combine knowledge gathered throughout different modules of the second and third period. Alongside my team we were able to combine, AI, computer vision, and industrial ROS to use the xarm for real life tracking of a automotive part, which I found pretty interesting since the challenge is based in a real industry problem proposed by General Motors. The training process using a point cloud based approach was something new and resulted difficult, however, with some research, and the help of the professor and Gustavo, we were able to cope with all the problems. I can firmly state that the experience obtained will be a great tool for my future professional career since it was useful not only to learn robotics, but also to learn teamwork, AI, industry problems and innovative solutions.

C. Erick

In this project, we successfully implemented the control and simulation of an XARM robotic arm using ROS2, RViz, and Gazebo. We achieved smooth integration between the hardware interface and virtual environment, enabling precise motion planning and execution of tasks within the simulated workspace. The use of RViz facilitated real-time visualization and monitoring of the arm's movements, while Gazebo provided a realistic physics-based environment for testing. This experience not only strengthened our technical skills in robotic systems and middleware but also highlighted the importance of simulation tools in the development and validation of robotic applications before deployment in real-world scenarios.

D. Cristian

Working on this project allowed me to better understand how robotics, artificial intelligence, and real-time systems come together to solve real industrial challenges. Throughout the development, I was especially involved in the integration and testing phases, where we applied AI and computer vision techniques to enable the XARM robotic arm to track an automotive component in a dynamic environment. One of the key takeaways for me was realizing how crucial it is to have both a strong theoretical foundation and hands-on practice—especially when working with ROS2, simulation environments like Gazebo, and real sensor data. While we faced several difficulties during the training of the detection model and system calibration, the collaborative effort of the team and guidance from our professor were essential to overcome them. This project not only improved my technical skills in robotics and automation but also emphasized the value of communication, adaptability, and critical thinking when facing complex, open-ended problems in an industry-like context. I'm confident that the experience will significantly contribute to my growth as a future

E. Eduardo

In conclusion, this project provided a comprehensive and highly enriching experience that merged theoretical knowledge with practical application in robotics, artificial intelligence, and real-time systems. By integrating technologies such as ROS2, RViz, Gazebo, and computer vision, we were able to simulate and control the XARM robotic arm to track an automotive part in a dynamic environment—tackling a real-world industrial challenge. Despite the difficulties encountered during the training of the detection model and the calibration process, the collaborative work of the team, combined with academic support, allowed us to successfully overcome these obstacles. This experience not only enhanced my technical skills in robotics, AI, and system integration, but also strengthened my ability to work in a team, solve complex problems, and adapt to the demands of real industrial environments. I firmly believe that what I learned during

this project will be a valuable asset for my future professional career.

F. Mauricio

This project represented a unique opportunity to bridge theoretical knowledge with hands-on practice in robotics, artificial intelligence, and real-time system integration. By working with ROS2, RViz, Gazebo, and computer vision, our team successfully developed a functional solution to a real-world industrial challenge—enabling the XARM robotic arm to track an automotive component, as proposed by General Motors. Beyond the technical aspects, the experience reinforced the importance of collaboration, adaptability, and continuous learning. Despite initial challenges in training the model and calibrating the system, we overcame them through research, teamwork, and academic guidance. Overall, this project not only deepened my understanding of robotic systems and AI applications but also strengthened essential skills that I will carry into my future career in the industry.

APPENDIX A

MODEL AND DATASET REPOSITORY

The complete resources used for the training and deployment of the AI model are available at the following Google Drive link:

<https://drive.google.com/drive/folders/13EHdk1zJ5d9cUkL5tZSNeCi64MtjU-G6?usp=sharing>

This folder includes:

- The full training and testing dataset consisting of 600 point cloud files (.ply) obtained from scanning the object using the Azure Kinect DK and .stl files provided by the lab.
- The Google Colab notebook used to process the data and train the PointNet model.
- The final trained Keras model (.h5) used during inference in the ROS 2 node.

These resources are shared for transparency and reproducibility, and they reflect the data pipeline and training process discussed throughout this paper.

APPENDIX B ROS2 NODE CODE

The complete ROS2 node code for controlling the XARM VI robotic arm is provided here.

[https://github.com/raymundol40/
Deep-Learning-Driven-XARM-VI-
Robotic-Arm-for-Automated-
Automotive-Part-Handling](https://github.com/raymundol40/Deep-Learning-Driven-XARM-VI-Robotic-Arm-for-Automated-Automotive-Part-Handling)

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Dr. Luis Alberto Muñoz Ubando for his guidance and support throughout the course and the development of this project. His expertise in robotics and consistent feedback were instrumental in shaping the technical direction and success of our work.

We also extend our special thanks to Gustavo De Los Ríos Alatorre for generously providing the 3D models, datasets, and valuable insights during the implementation process. His contributions played a key role in enabling the training and validation of the PointNet model.

Their support and mentorship were fundamental in achieving the results presented in this work.

REFERENCES

- [1] G. De Los Ríos Alatorre, *3-D Detection & Tracking for Semi-deformable Objects*, M.S. thesis, School of Engineering and Sciences, Tecnológico de Monterrey, Monterrey, Nuevo León, Mexico, May 2024.
- [2] Joseph Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2021.
- [3] R. Brown, *ROS2 Humble: A Guide to Robot Operating System 2*, O'Reilly Media, 2022.
- [4] S. Siciliano, L. Sciacivco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009.
- [5] M. F. Gennari, *Kinematics and Dynamics of Robotic Manipulation*, John Wiley & Sons, 2017.
- [6] M. Lippiello, A. Siciliano, and D. Siciliano, *Inverse Kinematics and Trajectory Planning for Robotic Manipulation*, Springer, 2018.
- [7] P. R. P. Nunes, D. R. Souza, and D. C. da Silva, *Control of Robot Manipulators: Theory and Applications*, Springer, 2020.
- [8] S. Koenig and A. Howard, *Design and Use of Gazebo: A Robot Simulation Framework*, Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2004.
- [9] A. Redmon, *YOLOv4: Optimal Speed and Accuracy of Object Detection*, arXiv preprint arXiv:2004.10934, 2020.
- [10] R. R. Murphy, *Introduction to AI Robotics*, The MIT Press, 2000.
- [11] M. Montemerlo et al., *The Stanford Racing Team's Entry in the Urban Challenge*, Journal of Field Robotics, 2008.
- [12] M. Lippiello, L. Rocco, and M. Siciliano, *Machine Learning Algorithms for Robotic Manipulation: A Survey*, Journal of Robotics and Automation, 2015.
- [13] E. S. S. V. Lippiello, *ROS2 Control: A Flexible and Modular Framework for Robot Control*, Springer, 2021.
- [14] B. Siciliano, L. Sciacivco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2016.